

SÉCURITÉ WEB

INTRODUCTION

- ▶ Pourquoi parler de **sécurité** aujourd'hui ?
- ▶ Un sujet qui nous concerne directement, **développeurs Symfony/PHP**

CE QUI PEUT ARRIVER



Détournement de site : redirection fauleuse, defacement



Publication de données confidentielles : fuite de base (identifiants, emails, IBAN)



Abus de ressources : cryptomining ou spam via serveur compromis



Vol d'identité : usurpation de compte, session volée

LA SÉCURITÉ, UN CYCLE

- ▶ **Conception** : anticiper qui accède à quoi
- ▶ **Développement** : bonnes pratiques, revue de code (CS Fixer/PHPStan)
- ▶ **Maintenance** : mises à jour, audits (Dependabot/composer audit)

LES GRANDES FAMILLES DE RISQUES ACTUELS

- ▶ Classement OWASP Top 10:2025
- ▶ Accès (A01)
- ▶ Configuration (A02)
- ▶ Chaîne d'approvisionnement (A03)

Source : <https://owasp.org/Top10/2025/>

LES PAGES WEB ET XSS

Sommaire & thématiques abordées

- ▶ **XSS** : principe et protections
- ▶ **CSRF** : principe et contre-mesures
- ▶ **Formulaires** : failles de validation et upload
- ▶ **Cookies et sessions** : risques et bonnes pratiques Symfony

XSS, C'EST QUOI

- ▶ **Cross-Site Scripting**

- ▶ Injection de code malveillant (souvent JavaScript) dans une page web
- ▶ Exécuté dans le navigateur d'une victime, pas sur le serveur

- ▶ 3 types : **Stocké, Réfléchi, DOM-based**

SYMFONY PROTÈGE DÉJÀ, EN PARTIE !

Twig échappe
automatiquement
tout output
(**autoescape** activé
par défaut)

Le filtre **raw**
désactive cette
protection : à
utiliser avec
prudence

Symfony ≠
immunité totale

LES ZONES À RISQUE MALGRÉ TWIG

- ▶ Contenu WYSIWYG / éditeur riche (`article.content|raw`)
- ▶ JavaScript inline avec variables non échappées pour ce contexte
- ▶ Attributs HTML dynamiques

LE XSS STOCKÉ

- ▶ Le payload malveillant n'est pas envoyé directement à la victime, il est stocké en base
- ▶ Toute personne qui consulte la donnée exécute le script, sans avoir rien cliqué
 - ▶ **Visiter la page suffit**, son effet se propage automatiquement à toute visite sur la page infectée
 - ▶ Chaque visiteur exécute le script, y compris des administrateurs
- ▶ Le vecteur classique : commentaire, avis client, profil utilisateur, message de contact
 - ▶ traiter toute donnée venant de la base comme non fiable à l'affichage

CONTENT SECURITY POLICY (CSP)

- ▶ Une couche de défense supplémentaire, indépendante du code
- ▶ Le navigateur bloque les scripts non autorisés, même si un XSS a été injecté
- ▶ Configurable dans Symfony !

CSRF, C'EST QUOI

- ▶ Cross-site Request Forgery
- ▶ L'attaquant fait exécuter une action à votre place, sans que vous vous en rendiez compte
- ▶ Exploite le fait que le navigateur envoie automatiquement les cookies de session
- ▶ Scénario type : virement bancaire, changement de mot de passe, suppression de compte

ERREURS SUR LE GET

- ▶ Erreur classique : une action sensible déclenchée par une **requête GET**
- ▶ Un simple lien ou une image peut suffire à déclencher l'attaque !
- ▶ Règle : toute action qui modifie une donnée doit être en **POST/PUT/DELETE**, jamais en GET

LE TOKEN CSRF, SON RÔLE

Un jeton unique généré par le serveur, injecté dans chaque formulaire

Vérifié à la soumission : **si absent ou invalide, la requête est rejetée**

Symfony l'active automatiquement pour tous les formulaires du composant Form

SOMMAIRE : COOKIES ET SESSIONS



COOKIES : PRINCIPES ET
RISQUES, MANIPULATION
JAVASCRIPT



SESSIONS : COOKIE VS HEADER,
VOL DE SESSION



LE SECURITY BUNDLE SYMFONY
COUVRE DÉJÀ L'ESSENTIEL !

PROTÉGER LES COOKIES

- ▶ Sans attribut, un cookie est lisible en JavaScript et transmissible en clair
- ▶ 3 attributs à connaître :
 - ▶ **HttpOnly** : empêche `document.cookie` de lire le cookie en JavaScript
 - ▶ **Secure** : interdit l'envoi du cookie sur une connexion HTTP non chiffrée
 - ▶ **SameSite** : contrôle l'envoi du cookie dans les requêtes cross-site
- ▶ Chacun bloque un vecteur d'attaque différent

HEADER HTTP : « Set-Cookie »

- ▶ Utilisation du « **Set-Cookie** »
 - ▶ En-tête de réponse HTTP envoyé par le serveur au navigateur
 - ▶ Information mémorisée par le navigateur, un ID de session, qui sera renvoyé à chaque requête afin de savoir qui est connecté
 - ▶ C'est ici que va circuler « **_security_main** », l'ID de session de connexion de Symfony

```
Set-Cookie: _security_main=abc123; Secure; HttpOnly; SameSite=Lax
```

LA SESSION

- ▶ Après connexion, le serveur crée une session contenant le **UsernamePasswordToken** sérialisé
 - ▶ Session « **_security_main** », où main est le nom du firewall dans le **security.yaml**
- ▶ Ce token est l'utilisateur pour toutes les requêtes suivantes
- ▶ Le voler, c'est usurper l'identité sans avoir besoin du mot de passe
 - ▶ Inutile de casser un mot de passe si on peut voler ce token

TRANSPORTER UN TOKEN D'AUTHENTIFICATION

- ▶ Mode cookie : le navigateur envoie le token automatiquement à chaque requête
 - ▶ Adapté à une application web classique avec rendu serveur (**Symfony/Twig**)
 - ▶ C'est le navigateur qui gère l'envoi, avec les protections **HttpOnly**, **Secure** et **SameSite**
- ▶ Mode header : le client doit explicitement joindre le token (ex : « **Authorization: bearer** »)
 - ▶ Adapté à une API (frontend + backend)
 - ▶ **localStorage** / **cookie**
- ▶ Le choix dépend du type d'application

VOLER UN TOKEN DE SESSION

- ▶ Via XSS : lecture du cookie non protégé en Javascript
 - ▶ Injecté via **document.cookie**, le token est envoyé vers un serveur connu de l'attaquant
 - ▶ **HttpOnly** bloque cette attaque
- ▶ Vol via sniffing du réseau : connexion non chiffrée (sans HTTPS)
 - ▶ Un attaquant sur le même réseau intercepte le token en clair
 - ▶ **Secure** bloque cette attaque
- ▶ Fixation de session : imposer un ID de session connu à la victime avant sa connexion
 - ▶ Via URL piégée, une fois l'utilisateur connecté on récupère le token

SYMFONY : FRAMEWORK.YAML

- Protections mises en place

SYMFONY : FRAMEWORK.YAML

- ▶ Par défaut Symfony utilise la valeur « **session: true** », qui ne rend pas visible la politique de sécurité appliquée au cookie. Elle prend les valeurs suivantes :
 - ▶ **HttpOnly: true**
 - ▶ **Secure** : dépends de la version et du php.ini, souvent « **auto** »
 - ▶ **SameSite** : généralement **lax**
- ▶ On peut aussi expliciter les comportements explicitement :

```
framework:  
  session: true
```

```
framework:  
  session:  
    cookie_secure: true  
    cookie_httponly: true  
    cookie_samesite: lax
```

SYMFONY : REMEMBER ME

La fonctionnalité « rester connecté » : un cookie longue durée, séparé de la session



Si fait maison : token statique facilement réutilisable en cas de vol



Symfony : rotation de token à chaque utilisation
Configuration dans `security.yaml`, tout est

BROKEN ACCESS CONTROL

- ▶ Principe : l'application vérifie qu'on est authentifié, mais pas qu'on a le droit d'accéder à **CETTE ressource précise**
- ▶ Classement **OWASP A01:2025** : première cause de vulnérabilité, devant les injections
- ▶ **IDOR** (Insecure Direct Object Reference) : la forme la plus fréquente et la plus simple à exploiter
 - ▶ Il faut distinguer la différence entre l'authentification (qui vous êtes) et l'autorisation (avez-vous le droit d'accès à cette ressource)
 - ▶ Exemple : « **GET /api/orders/1234** » ; si on change l'ID passé en paramètre, ici « 123 », et qu'aucune vérification n'est faite sur cette route, alors on peut accéder à des données d'un autre client
 - ▶ Exemple : « **?file=invoice_1000.pdf** » ; si on change le nom du fichier PDF, on peut accéder à un document d'un autre utilisateur

VOTERS SYMFONY

- ▶ Une classe dédiée par type de ressource, réutilisable partout dans l'application
 - ▶ Extends « **Voter** »
- ▶ Appelé via « **isGranted()** » ou « **denyAccessUnlessGranted()** »
 - ▶ Utilisation d'attribut PHP « **#[IsGranted()]** » depuis Symfony 6.2
- ▶ Lorsque l'on fait un « **isGranted()** », tous les **Voters** enregistrés sont appelés un à un
 - ▶ « **supports()** » : « Ce Voter doit-il se prononcer sur cette demande ? »
 - ▶ « **voteOnAttribute()** » : appelé uniquement si « **supports()** » a retourné « **true** »

CRÉATION D'UN VOTER

```
class OrderVoter extends Voter
{
    public const VIEW = 'VIEW';

    protected function supports(string $attribute, mixed $subject): bool
    {
        return $attribute === self::VIEW && $subject instanceof Order;
    }

    protected function voteOnAttribute(string $attribute, mixed $subject, TokenInterface $token): bool
    {
        $user = $token->getUser();
        return $subject->getOwner() === $user;
    }
}
```

UTILISATION D'UN VOTER

```
#[IsGranted(OrderVoter::VIEW, subject: 'post')]
```

- ▶ Le premier paramètre est l'« **attribute** »
- ▶ Le deuxième paramètre est le « **subject** », l'entité dont on souhaite vérifier l'accès
- ▶ Même dans le cas d'un seul attribut, comme ici **VIEW**, on centralise la logique à un seul endroit. De plus cela permet de mieux gérer les évolutions, comme un cas « **EDIT** » ou « **DELETE** »

SÉCURITÉ DES API

▶ JWT

▶ Quota

SÉCURITÉ DES API : JWT

- ▶ Durée de vie courte, en général le token d'accès est valide de 5 à 15 minutes
 - ▶ Réduit la marge de manœuvre s'il est intercepté
- ▶ **Jamais de données sensibles** dans le payload
 - ▶ Déchiffvable, on communique général l'identifiant utilisateur et les dates de validité
 - ▶ Un token répond à un besoin : « **qui suis-je** », pas « **que puis je faire** »
- ▶ Stockage : initialement en **localStorage**, préférer cookie en **HttpOnly**
 - ▶ N'importe quel script Javascript, ou XSS peut y accéder
- ▶ Révocation : empêcher qu'un token normalement valide, soit accepté

QUOTA ET RATE LIMIT

- ▶ Même une faille bien protégée reste exploitable en masse, sans limite de requêtes
- ▶ **RateLimiter** Symfony / API Platform : throttling natif, configurable par route
 - ▶ « **composer require symfony/rate-limiter** »
 - ▶ Configuration dans le « **rate_limiter.yaml** »
 - ▶ Injection dans les contrôleurs via « **RateLimiterFactory** »
 - ▶ Appliquer ces changements sur des routes de « **login** » ou « **contact** » par exemple

QUOTA ET RATE LIMIT : IMPLÉMENTATION

- Exemple de « `rate_limiter.yaml` »

```
# config/packages/rate_limiter.yaml
framework:
  rate_limiter:
    anonymous_api:
      policy: 'fixed_window'
      limit: 100
      interval: '60 minutes'
```

QUOTA ET RATE LIMIT : IMPLÉMENTATION

- Exemple dans un contrôleur :

```
public function index(RateLimiterFactory $anonymousApiLimiter, Request $request)
{
    $limiter = $anonymousApiLimiter->create($request->getClientIp());

    if (false === $limiter->consume(1)->isAccepted()) {
        throw new TooManyRequestsHttpException();
    }
    // ... traitement normal de la requête
}
```

LOGIN_THROTTLING

- ▶ Exemple de `security.yaml` :

```
security:
  firewalls:
    main:
      login_throttling:
        max_attempts: 5
        interval: '5 minutes'
```

- ▶ Le « `login_throttling` » permet d'avoir :
 - ▶ un limiteur **par couple identifiant + IP**, avec `max_attempts` (ici 5)

PHP.INI

- ▶ Affichage d'erreurs
- ▶ Environnements

PHP.INI

- ▶ **display_errors= off** : n'affiche jamais les erreurs PHP au visiteur (chemin de fichiers ou requêtes SQL)
 - ▶ Affichera les chemins de fichiers ou noms de fonctions dans les messages d'erreurs
- ▶ **log_errors = on** : les erreurs doivent être tracées côté serveur
- ▶ **expose_php = off** : cache la version de PHP dans les entêtes HTTP
- ▶ **allow_url_fopen = off & allow_url_include = off** : empêchant d'inclure un fichier distant comme du code local
 - ▶ Directement lié à la **Remote File Inclusion (RFI)**

PHP.INI & SESSIONS

- ▶ **session.cookie_httponly, session.cookie_secure** : même configuration des cookies, mais cette fois côté serveur
- ▶ **session.use_strict_mode = on** : on rejette les identifiants de session non initialisés par le serveur
- ▶ **disable_functions** : désactive les fonctions « dangereuses » (**exec, system, passthru** ou encore **shell_exec**)
 - ▶ Peut aussi empêcher des besoins métiers légitimes, comme la génération de PDF !
- ▶ **open_basedir** : restreint l'accès aux fichiers au seul répertoire de l'application
 - ▶ Limite l'utilisation des fonctions de fichiers à un répertoire précis

PROTECTIONS DES SCRIPTS

- ▶ Exécution distante (**RCE**, Remote Code Execution) est la conséquence la plus grave d'une chaîne de failles combinées
- ▶ Upload de fichier + absence de filtrage d'extension = dépôt d'un script PHP exécutable
- ▶ **RFI** (Remote File Inclusion) : inclusion d'un fichier hébergé chez l'attaquant
- ▶ Contre-mesure structurelle : séparer le répertoire d'upload du répertoire exécutable par le serveur web
 - ▶ Même si on arrive à forcer l'upload d'un script **shell.php**, déguisé en image, alors le script ne pourra pas être exécuter

SYMFONY : CONFIGURATION PAR ENV

- ▶ **dev/prod** : deux comportements différents, notamment sur les erreurs affichées
 - ▶ Similaire à **display_errors = off** pour la **pro**, mais gérer côté framework
- ▶ **APP_ENV** et **APP_DEBUG** pilotent ce comportement, jamais à activer en prod
 - ▶ En prod il faut s'assurer que **APP_DEBUG=0**
- ▶ **Secrets Vault** : chiffre les données sensibles (clés API, mot de passe DB) dans le dépôt GIT
- ▶ Paire de clés (publique/privée) : la clé privée de déchiffrement ne doit jamais être committée

ENTÊTES HTTP DE SÉCURITÉ

- ▶ **CSP** (Content-Security Policy) : contrôle des sources de scripts autorisées
- ▶ **HSTS** (Strict-Transport-Security) : force le **HTTPS**, même si l'utilisateur tape **http://**
- ▶ **X-Frame-Options** : empêche d'intégrer le site dans une iframe malveillante (clickjacking)
- ▶ A02:2025 dans le classement OWASP : ces outils de configurations sont la 2^{ème} cause de vulnérabilité

BASE DE DONNÉES



FAILLES POTENTIELLES



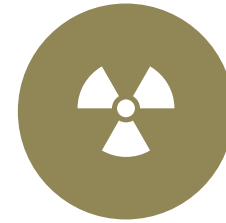
INJECTION SQL



REQUÊTES PARAMÉTRÉES
ET PROCÉDURES
STOCKÉES



DOCTRINE ORM &
QUERYBUILDER



RISQUES MALGRÉ L'ORM

BASE DE DONNÉES : FAILLES POTENTIELLES

- ▶ Accès direct exposé : port MySQL/PostgreSQL accessible depuis internet
 - ▶ Devrait être accessible depuis un réseau interne
- ▶ Compte administrateur par défaut, mot de passe faible ou absent
 - ▶ Même en environnement de test, il est recommandé de ne pas utiliser un mot de passe trivial
- ▶ Sauvegardes non chiffrées, stockées sans contrôle d'accès
 - ▶ Les sauvegardes sont souvent négligées, mais il est facile de récupérer l'intégralité de la base via celles-ci
- ▶ Privilèges excessifs : l'application se connecte avec un compte ayant tous les droits
 - ▶ Il est recommandé de n'autoriser que des droits strictement nécessaires (SELECT/INSERT/UPDATE) sur ses propres tables

INJECTION SQL

- ▶ Une donnée utilisateur concaténée directement dans une requête SQL
- ▶ L'attaquant injecte du code SQL qui modifie la logique de la requête
- ▶ Exemple classique : contournement d'authentification
 - ▶ `$query = "SELECT * FROM users WHERE email = '$email' AND password = '$password'";`
 - ▶ L'attaquant saisit : `' OR '1' = '1` comme mot de passe
 - ▶ `SELECT * FROM users WHERE email = 'admin@site.com' AND password = " OR '1'='1'`
- ▶ Impact possible : lecture de toute la base, modification, suppression, parfois exécution de commandes serveur

REQUÊTES PARAMÉTRÉES ET PROCÉDURES STOCKÉES

- ▶ Requête paramétrée : la donnée utilisateur n'est jamais insérée directement dans la chaîne SQL
- ▶ Le moteur SQL distingue structure de la requête et données, quelle que soit leur contenu
- ▶ Procédures stockées : logique SQL prédéfinie côté serveur, appelée avec des paramètres
 - ▶ Une procédure stockée n'est pas automatiquement sécurisée, elle construit elle-même une requête dynamique en concaténant les paramètres reçus
- ▶ Limite des procédures stockées : ne protègent pas si elles concatènent elles-mêmes les paramètres reçus

DOCTRINE ORM & QUERYBUILDER

- ▶ Doctrine paramètre automatiquement toutes les requêtes générées via ses méthodes standards
- ▶ `find()`, `findBy()`, `QueryBuilder` avec `setParameter` : tout est protégé nativement

```
$qb = $this->createQueryBuilder('u');  
$qb->where('u.email = :email')  
    ->setParameter('email', $email);
```

RISQUES MALGRÉ L'ORM

- ▶ Requêtes SQL natives : `executeQuery` ou `Connection::query`, n'offre aucune protection automatique
- ▶ Concaténation résiduelle dans le **DQL/QueryBuilder** : construire dynamiquement une clause avec une variable
 - ▶ `$conn->executeQuery("SELECT * FROM users WHERE email = " . $email . " « ' »");`
 - ▶ `$conn->executeQuery("SELECT * FROM users WHERE email = ?", [$email]);`
 - ▶ `$qb->where("u.email = " . $email . "");`
 - ▶ `$qb->where('u.email = :email')->setParameter('email', $email);`
- ▶ Noms de colonnes ou de tables dynamiques : non paramétrables en SQL
- ▶ L'ORM protège la donnée, pas la structure de la requête

SÉCURISER L'UTILISATION DES EXTENSIONS

Injection d'en-têtes email via formulaire de contact

Filtrage et Symfony Mailer

SSRF

INJECTION D'EN-TÊTES EMAIL VIA FORMULAIRE DE CONTACT

- ▶ Une donnée saisie dans un champ (nom, email) est insérée directement dans les en-têtes d'un email
- ▶ L'attaquant y injecte des retours à la ligne (`\r\n`) pour ajouter ses propres en-têtes
 - ▶ `attacker@evil.com\r\nBcc: victime1@mail.com,victime2@mail.com,victime3@mail.com`
 - ▶ Cela envoie le mail en copie cachée à multiples destinataires, ce qui peut faire que votre domaine soit blacklisté
- ▶ Conséquences : le formulaire devient une passerelle d'envoi de spam massif, depuis votre domaine
- ▶ Vecteur historique : la fonction native PHP `mail()`

FILTRAGE ET SYMFONY MAILER

- ▶ Symfony n'utilise pas la fonction native « **mail()** » directement avec des données utilisateur non filtrées
- ▶ Il rejette les \r ou \n dans les champs
- ▶ Symfony Mailer : construit les en-têtes via une API structurée, pas de concaténation manuelle possible

```
$email = (new Email())  
    ->from('contact@monsite.com') // adresse fixe, généralement depuis un .env  
    ->to('support@monsite.com')  
    ->subject('Nouveau message de contact')  
    ->text($userMessage);        // le contenu utilisateur reste dans le corps
```

PRINCIPE DU SSRF

- ▶ **OWASP A10, SSRF** (Server-Side Requ^eSt Forgery) : il s'agit d'une vulnérabilité de sécurité qui permet à un attaquant de forcer un serveur vulnérable à envoyer des requêtes HTTP vers des destinations non intentionnelles.
- ▶ Cela fonctionne comme un cheval de Troie », où l'on se sert du serveur pour envoyer une requête malveillante depuis le serveur
- ▶ Le serveur devient un relais permettant d'atteindre des ressources normalement inaccessibles depuis l'extérieur

CONTRE-MESURES CONTRE LE SSRF

- ▶ Liste blanche stricte des domaines/hôtes autorisés, jamais de liste noire
 - ▶ Il est plus facile, via des techniques d'encodage d'URL ou de redirection de contourner une liste noire
- ▶ Bloquer les plages d'adresses internes (172.0.0.1, 172.16.31.xx, etc)
 - ▶ Résoudre le nom de domaine en IP et vérifier cette IP avant chaque connexion, y compris après redirection
- ▶ En environnement cloud : imposer **IMDSv2 (AWS)** qui neutralise les requêtes SSRF simples
 - ▶ **IMDSv2** exige une requête **PUT** avec un jeton de session avant tout accès aux métadonnées

ATTAQUES TRANSVERSES, SUPPLY CHAIN ET AUDIT

- ▶ Attaques qui ne visent pas une faille de code, mais le comportement global de l'application
- ▶ Brute-force
- ▶ Supply chain
- ▶ Audit

BRUTE-FORCE

- ▶ Tester systématiquement un grand nombre de combinaisons de mots de passe jusqu'à trouver la bonne
 - ▶ Test de mot de passe au hasard ou via un dictionnaire contre un seul compte ciblé
- ▶ Variante « **credential stuffing** » : réutiliser des identifiants volés sur d'autres sites
 - ▶ Réutiliser une combinaison d'email/mot de passe volée lors d'une précédente fuite de données, en pariant que l'utilisateur réutilise le même mot de passe partout
- ▶ Signal à surveiller : de nombreux échecs de connexion sur un même compte, ou depuis de nombreuses IP différentes vers un même compte

BRUTE-FORCE : CONTRE-MESURES

- ▶ « **RateLimiter** » de Symfony sur la route de « **login** »
 - ▶ Très bien adapté à ce cas de figure, seulement sur la route de **login**
- ▶ **CAPTCHA** ou vérification supplémentaire après plusieurs échecs
- ▶ Authentification à deux facteurs (**2FA**) : rend le brute-force inutile même en cas de mot de passe trouvé
- ▶ Verrouillage temporaire de compte après plusieurs échecs répétés, avec notification à l'utilisateur

PHISHING

- ▶ Ne cible pas une faille technique de l'application, **mais la confiance de l'utilisateur**
 - ▶ Ce n'est pas la faute du développeur, mais on peut malgré tout mettre en place des choses pour limiter cela
- ▶ Exemple : email ou page frauduleuse imitant l'application réelle pour voler des identifiants
- ▶ Ce que l'on peut faire malgré tout : cohérence des emails envoyés, 2FA, domaine et certificats clairs
 - ▶ Cohérence des mails, toujours depuis le même domaine et la même structure
 - ▶ 2FA : empêchera toujours la finalisation de la connexion, même avec des identifiants usurpés
 - ▶ Domaine et certificats clairs : éviter les sous-domaines à rallonge ou peu lisibles pour les pages d'authentification

DoS : QUOTAS

- ▶ Objectif de l'attaquant : saturer les ressources serveur pour rendre le service indisponible
- ▶ **RateLimiter** appliqué à l'échelle globale de l'API
- ▶ **API Platform** : quotas configurables nativement par ressource ou par utilisateur
 - ▶ Ce n'est pas « clé en main », c'est quelque chose à implémenter via **RateLimiter** !
 - ▶ Utiliser un **EventListener Symfony** global
 - ▶ Un state Provider personnalisé (une classe héritant de **ProviderInterface**)

CHIFFREMENT ET SIGNATURE

- ▶ Chiffrement : garantit la confidentialité, seul le destinataire légitime peut lire le contenu
 - ▶ « Qui peut lire ce contenu ? » ; le message devient illisible sans la clé de déchiffrement
- ▶ Signature : garantit l'authenticité et l'intégrité, prouve qui a créé la donnée et qu'elle n'a pas été modifiée
 - ▶ « Qui a créé ce contenu, et a-t-il été modifié depuis ? » ; le message reste lisible, mais on peut vérifier son origine et son intégrité
- ▶ Un JWT est signé, pas chiffré : son contenu reste lisible par tous

SUPPLY CHAIN

- ▶ « **La chaîne d'approvisionnement** » c'est l'ensemble du code que l'on n'a pas écrit, mais que l'application exécute quand même, comme des dépendances installées via **composer**
- ▶ On fait confiance à ce code, que l'on va exécuter en production avec les mêmes droits que notre propre code
- ▶ Auparavant, la sécurité se concentrait sur le code qu'on écrit soi-même, maintenant la majorité du code qui tourne dans une application vient de dépendances tierces. Un seul paquet compromis peut affecter des milliers d'applications qui l'utilisent, en une seule mise à jour

EXTRA

- ▶ « RateLimiter » Vue.js
- ▶ Route POST
 - ▶ Token JWT en cookie ou localStorage ?
 - ▶ Prévoir les contraintes de validations
- ▶ Voters

« RateLimiter » Vue.js

- ▶ **useDebounceFn** : retarde l'exécution jusqu'à ce que les appels cessent, pendant un délai donné. Chaque nouvel appel relance le chronomètre
 - ▶ Adapté aux barres de recherche, on envoie la requête que lorsque l'utilisateur a finit de taper
- ▶ **useThrottleFn** : exécute la requête directement et ignore les suivantes pendant un délai
 - ▶ Adapté au scroll continu, protéger un bouton contre le spam
- ▶ **:disabled="loading"** : tout simplement initialiser un boolean à false avant la requête HTTP, et tant qu'elle est envoyée, on disable le bouton. On est sûr qu'une seule requête parte

ROUTE POST : ROUTE UNIQUE

- ▶ Rapide à mettre en place, il faut cependant ajouter :
 - ▶ Classe de correspondance entre alias, envoyé dans le POST avec le **FormType** et les attributs **Voter**
- ▶ Aucune modification côté front !
- ▶ Point faible : **tout le risque est concentré au même endroit !**
 - ▶ « Contrôleur dieu »
 - ▶ Cas particuliers qui s'accumulent
 - ▶ Traitement spécifique après la création ? Email ?
 - ▶ Tests compliqués ?

ROUTE POST : ROUTE PAR ENTITÉ

- ▶ Plus lent à mettre en place, en fonction du nombre d'entité...
 - ▶ On peut malgré tout le faire de **manière progressive**
- ▶ Modifications des appels HTTP côté Front
- ▶ Points forts :
 - ▶ Chaque route a sa logique propre, ses « **#IsGranted** », son FormType
 - ▶ Tests faciles à mettre en place par route

VOTERS

- ▶ Un Voter répond au besoin « **cet utilisateur peut-il faire X sur cet objet ?** »
- ▶ Gestion des « **NEW** » sur l'enfant, depuis le parent !
 - ▶ **CREATE_PROMOTION** depuis le **FormationVoter**
- ▶ Routes hiérarchiques ?
 - ▶ `/institution/{i}/composant/{c}/formation/{f}/promotion/new`
 - ▶ Le parent de **promotion** (**Formation**) est l'objet récupéré depuis la route
 - ▶ **PermissionResolver** : fonctionne si le parent a le droit sur tout ses enfants

GITHUB

- ▶ <https://github.com/TyTy-cf/2026-websecurity>